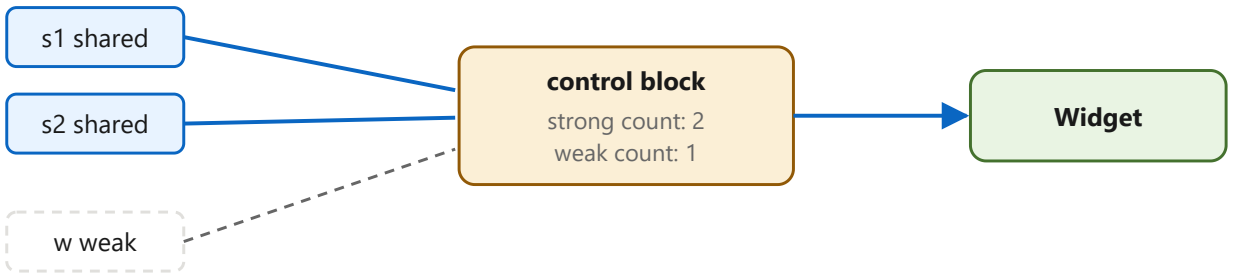
shared_ptr and weak_ptr

Shared ownership: `shared_ptr` and the control block, reference cycles and `weak_ptr`, and the pitfalls. Assumes unique_ptr, which covers ownership and the rule that a smart pointer is only for objects you own.

`shared_ptr` and the control block



Object freed when strong count hits 0. weak_ptr.lock() gives a shared_ptr or null.

unique_ptr has no control block: zero overhead, one owner.

```
// one allocation for object + control block
auto s1 = std::make_shared<Widget>(2, "b");
// two owners, use_count() == 2
auto s2 = s1;
```

- A `shared_ptr` is **two pointers**: to the object and to a **control block** holding the strong count, the weak count and the deleter.
- `make_shared` allocates object and control block together (one allocation). `shared_ptr<T>(new T)` allocates twice.
- The deleter is not part of the `shared_ptr` type (unlike `unique_ptr`): type-erased in the control block.
- The counts are **atomic**: copying or destroying `shared_ptr`s on several threads is safe. The **object** is not protected, and changing the same `shared_ptr` variable from two threads is a race.
- Copying costs an atomic increment. On hot paths pass `const shared_ptr&` or the object itself, not a copy per call.

Reference cycles and `weak_ptr`

If `A` holds a `shared_ptr<B>` and `B` holds a `shared_ptr<A>`, each keeps the other alive: **neither is ever destroyed**. Compiled: after the scope, both objects were still alive (leaked). Make the back link (child to parent) a `weak_ptr`: both were destroyed.

```
struct Node { std::shared_ptr<Owner> parent; };        // cycle: leak
// weak back-pointer: freed
struct WNode { std::weak_ptr<OwnerW> parent; };

// observe only: does not change use_count
std::weak_ptr<Widget> w = s1;
// lock() gives a shared_ptr or null
if (auto sp = w.lock()) { /* still alive: use sp for as long as you hold it
*/ }
// true once every owner is gone
w.expired();
```

- `lock()` is the only way to use the object: it gives a `shared_ptr` that keeps it alive while you work, or null.
- Typical uses: parent links, caches, observer lists that must not keep subscribers alive.
- A `weak_ptr` keeps the **control block** alive (not the object) until the last weak reference goes.

Pitfalls and `enable_shared_from_this`

ONE CONTROL BLOCK PER OBJECT

```
auto raw = new Widget;
std::shared_ptr<Widget> a(raw);
// a second, independent control block: double delete!
std::shared_ptr<Widget> b(raw);
```

Never build a second `shared_ptr` from a raw pointer or from `this`. Copy an existing one.

When a member function needs a `shared_ptr` to its own object (to register itself, to hand to a callback), inherit `std::enable_shared_from_this` and call `shared_from_this()`:

```
struct Session : std::enable_shared_from_this<Session> {
    // a safe second owner, SAME control block
    std::shared_ptr<Session> self() { return shared_from_this(); }
};
auto s = std::make_shared<Session>();
auto t = s->self();                    // use_count() == 2
// throws std::bad_weak_ptr: no shared_ptr owns it (compiled)
Session raw; raw.self();
```

PITFALL	FIX
`shared_ptr` cycle	`weak_ptr` for the back link
`shared_ptr` from `this` or a raw pointer	`enable_shared_from_this`
`shared_ptr` by default "to be safe"	`unique_ptr`: you can always convert to shared later
storing `.get()` somewhere that outlives the owner	keep the smart pointer, or a `weak_ptr`
`delete` on a `.get()` result	never: the smart pointer will delete it again

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
shared_ptr and the control block	C++ Primer	12.1 Dynamic memory and smart pointers	479 to 504
	Programming: Principles and Practice	18.4 to 18.5 Resources and exceptions; resource-management pointers (RAII, unique_ptr)	542 to 553
	Effective Modern C++	Item 19, Use std::shared_ptr for shared-ownership resource management	143 to 152
	Effective Modern C++	Item 21, Prefer make_unique and make_shared to new	157 to 165
Reference cycles and weak_ptr	C++ Primer	12.1 Dynamic memory and smart pointers	479 to 504
	Effective Modern C++	Item 20, Use std::weak_ptr for shared_ptr-like pointers that can dangle	152 to 157